a) Rushan wants to develop a server where everything that is being requested is handled by creating a child process of the original server process. The child process tends to complete and gets terminated, but after a while, Rushan notices that many inactive processes that have completed their task remain in the process table and are clogging up the system over time. I. Identify what type of issue Rushan is facing with her system. II. State how it can be solved.

I. What issue is Rushan facing?

Rushan is facing the **Zombie Process problem**.

- When a child process finishes execution, it does **not fully disappear immediately**.
- It remains in the process table as a **zombie (defunct) process** until the parent process reads its exit status.
- Since the server keeps creating child processes but **does not clean them up**, these zombies accumulate and **clog the system**.

Without wait():

- The parent may terminate before the child.
- The child becomes an **orphan process**

II. How can this be solved?

The issue can be solved by ensuring the parent process properly handles child termination:

✔️ **Solution 1: Use wait() or waitpid()**

c) A parent process forks, creating a child process. After the fork

completes, the parent process creates a new shared memory segment and

writes the number 111 into it.

I. Mention in yes or no if the child process now attempts to access

that same shared memory segment, will it be able to see the

number 111?

II. Justify the reason behind the previous answer.

I. Answer

No.

II. Justification

After a fork(), the child process gets a **copy of the parent's address space at that exact moment**.

- The shared memory segment in this case is **created *after* the fork**.
- That means the child process was never **inherited or attached to that shared memory segment**.
- Shared memory is **not automatically visible** to other processes unless they explicitly attach to it (e.g., using shmget() + shmat() or similar).

So even though the parent writes **111**, the child:

- does **not know about that shared memory segment**, and
- therefore **cannot access or see the value 111**.

2.

CO1

a) Write the answers in the question paper.

Suppose in a particular scenario FCFS CPU scheduler is being used.

I. What significant drawback arises for short, I/O-bound processes

when a long, CPU-intensive process is scheduled first using the

Algorithm?

👉 In **FCFS (First Come First Serve)**:

- Processes are executed in the order they arrive
- If a **long CPU-intensive process comes first**, it holds the CPU for a long time

❗ **Problem:**

Short, I/O-bound processes must **wait for a long time**

📌 This causes:

- **High waiting time**

- **Poor response time**

👉 This situation is called:

Convoy Effect

✔ Meaning:

Many short processes get stuck behind one long process

II. How does the issue lead to inefficient use of the CPU?

This leads to inefficient CPU utilization because short I/O-bound processes are delayed from executing and performing I/O. As a result, the CPU may remain idle while processes are waiting for I/O, reducing overall system efficiency.

i) Let's consider a scenario where a total of 4 processes are available for

CPU allocation. Now, they have burst times like 4, 5, 6, 8. The OS is

configured to use Round-Robin scheduling with a time quantum of 10.

At this point, figure out the problem with this amount of time quantum.

Problem with the given time quantum:
The time quantum (10) is **too large** compared to the burst times (4, 5, 6, 8).

Explanation:
In Round-Robin scheduling, the time quantum should be small enough to allow fair sharing of CPU among processes. However, here:

- All processes have burst times **less than the time quantum (10)**
- So, each process will **finish execution in one turn** without being preempted

This makes Round-Robin behave like **First-Come, First-Served (FCFS)** scheduling.

i) You are given an array of 100 integers. You are required to calculate

the sum of these integers using a 10-core CPU. Identify what kind of

parallelism you need to implement to ensure that the operation is

performed most efficiently.

Answer:

To calculate the sum of 100 integers using a 10-core CPU efficiently, we should use **data parallelism**.

- **Method:** Divide the array into 10 equal parts (10 integers each).
- **Step 1:** Each core computes the sum of its assigned 10 integers independently.
- **Step 2:** Combine the 10 partial sums to get the final total.

This approach ensures all cores work simultaneously, minimizing computation time.

a) Write the answer in the question paper.

Suppose you are running a background data processing program that spawns a child process to handle file uploads. Suddenly, the parent process crashes before finishing and never calls wait() to check the child's exit status. Write what happens to the child process in this situation?

Answer:

If the parent process crashes before calling wait(), the child process becomes a **zombie after it finishes**.

- The child process **still exists in the process table** with its exit status, because no parent has collected it.
- If the original parent has completely terminated, the child is **adopted by the init process** (or a similar system process), which then **calls wait()** to clean it up, preventing it from remaining a zombie.

Adhara wants to build a media editing application where multiple

processes such as audio, video and effects processes work on the

same data simultaneously. Since the data needs to get accessed

and updated frequently, she needs a communication method that

ensures high speed with minimal kernel support.

I. Identify the suitable interprocess communication method

for Adhara in this scenario.

II. State the reason behind the selection.

Answer:

I. Suitable IPC method: Shared Memory

II. Reason:

- Multiple processes (audio, video, effects) need to **access and update the same data simultaneously**.
- **Shared memory allows all processes to read/write data directly** in a common memory space.
- It provides **high speed** because **data does not need to pass through the kernel**, unlike pipes or message queues.
- Minimizes kernel involvement and overhead, which is ideal for a **media editing application** with frequent, real-time updates.

a) Write the answer in the answer script.

Dipu has implemented two identical MLFQ schedulers. In the first implementation, new processes arrive at the queue having the highest priority level. In the second implementation, new processes arrive at the queue having the lowest priority level. Apart from the mentioned difference, all other operations of MLFQ are the same as regular MLFQs. A balanced number of I/O and CPU-bound processes arrive in the same order in both implementations of Dipu.

Explain how the different scheduling criteria would differ across both implementations.

Answer:

The two MLFQ implementations will behave differently due to **the initial priority of new processes**:

1. **New processes at highest priority:**
    - Processes start at the top queue, getting **immediate CPU access**.
    - **CPU-bound processes** may dominate CPU early, potentially delaying I/O-bound processes.
    - **I/O-bound processes** can quickly move back to the top after waiting, improving responsiveness.
    - Favours **interactive and short tasks**.
2. **New processes at lowest priority:**
    - Processes start at the bottom queue, getting **CPU only after higher-priority queues are empty**.
    - **CPU-bound processes** are gradually promoted, delaying their execution initially.
    - **I/O-bound processes** may also wait longer, reducing responsiveness.

    - Favours **long-running CPU-bound tasks** but can cause poor interactive performance.

Conclusion:

- Starting at **highest priority** benefits short, interactive jobs and improves responsiveness.
- Starting at **lowest priority** favours CPU-bound tasks but can increase waiting time for new processes, especially interactive ones.

c) Write the answer in the question paper.

You have designed an MLFQ with three total queues, the highest-priority queue being set with a time quantum of 50 ms, and the next two queues having time quantums of 200 ms and 500 ms, respectively. Now you are deciding on the promotion factors of the processes.

You have two options in mind:

a. If a process completes an I/O operation, it returns to the same level from which it came.

b. It will get promoted to a higher priority level upon completing an I/O operation.

You have an interactive program that repeatedly performs the following from the very beginning after entering the queue: a short burst (30 ms) of CPU usage, and then waits for I/O to retrieve user information.

Explain briefly with proper reasons whether there will be any differences in performance between the two choices for this process.

Answer:

For the interactive program with short CPU bursts (30 ms) followed by I/O:

1. **Option a (return to same level):**

    - The process will stay in the **highest-priority queue** because its CPU burst is less than the 50 ms quantum.
    - It gets CPU quickly after each I/O, resulting in **low waiting time and high responsiveness**.
2. **Option b (promote to higher level):**
    - Since the process is already in the **highest-priority queue**, promotion has **no effect**.
    - Performance remains the same as Option a.

Conclusion:

- For this short-burst interactive process, **there is no difference in performance** between the two promotion strategies.
- The process will always complete its CPU bursts quickly and respond immediately after I/O.

b) Write the answer in the question paper.

Explain whether it is always possible or not in a multicore system

during multithreading to speed up a program by n times if we use

n threads instead of one.

Answer:

It is **not always possible** to speed up a program by exactly n times in a multicore system, even if we use n threads.

Reasons:

1. **Amdahl's Law:**
    - If a portion of the program is **sequential and cannot be parallelized**, it limits the maximum speedup.
    - Example: If 10% of the program is sequential, the maximum speedup cannot exceed 10×, even with infinite threads.
2. **Overhead of multithreading:**
    - Creating threads, context switching, and synchronization introduce extra **time overhead**, reducing speedup.
3. **Resource contention:**
    - Multiple threads may compete for **shared memory, caches, or I/O**, slowing down performance.
4. **Load imbalance:**
    - If work is not perfectly divided among threads, some threads may finish earlier and others may idle.

c) Write the answer in the answer script.

Write the issues with proper logic that might arise if we use

recursive functions in a multi-threaded program.

Answer:

Using recursive functions in a multi-threaded program can cause:

1. **Stack overflow:** Each thread maintains its own stack; deep recursion may exceed stack size.
2. **Race conditions:** If recursive calls access **shared data** without proper synchronization, data corruption can occur.
3. **Increased overhead:** Many recursive calls across threads can increase **context switching and memory usage**, reducing performance.

d) Write the answer in the answer script.

The variable counter is a global variable that is shared by both thread-1 and thread-2. The initial value of counter is 0, temp1 and temp2 are local variables. Consider thread-1 and thread-2 are run at the same time.

are run at the same time.

Code run by thread-1	Code run by thread-2

`// unrelated statements`
`temp1 = counter;`
`temp1 += 1;`
`counter = temp1;`
`// unrelated statements`
	`// unrelated statements`
	`temp2 = counter;`
	`temp2 += 2;`
	`counter = temp2;`
	`// unrelated statements`

I. Identify what kind of problem can arise in this scenario and how it can be avoided.

II. Modify the codes of both thread-1 and thread-2 above by utilizing two binary semaphores to ensure that the final value of counter is exactly 2, after both threads have finished their execution. You have to mention the initial values of both semaphores.

I. Problem and Solution:

- The scenario can lead to a **race condition**, because both threads access and update the shared variable counter concurrently.
- **Solution:** Use **binary semaphores** to ensure **mutual exclusion** when accessing counter.

II. Modified Code Using Two Binary Semaphores

- Let **sem1** and **sem2** be binary semaphores.
- **Initial values:** sem1 = 1, sem2 = 1

b) Alex wants to develop such a platform where users from around the world can share information among themselves. His motive is to connect users under a server and use a dedicated storage to exchange information. In order to share information users require to send and receive data in the storage among each other. For his platform Alex is more concerned about the size of the storage over data transfer rate.

Identify with proper logic which communication method is suitable for Alex's platform?

Answer:

The suitable communication method is **Message Queue**.

Reason:

- Message queues use **kernel-managed storage (buffer/queue)** to store messages, which supports **larger data storage** compared to pipes.
- Communication is **asynchronous**, so users can send and receive data at different times using the shared storage.
- Since Alex is more concerned about **storage size rather than speed**, message queues are more appropriate than faster but limited methods like pipes.

a) Why do we need cooperating processes, and what are the two models of interprocess communication? Additionally, mention the strengths and weaknesses of each approach?

a) Cooperating processes & IPC models [3]

Why needed:

- To **share information**, improve **performance (parallelism)**, and support **modular system design**.

Two IPC models:

1. **Shared Memory**

    - **Strength:** Very fast (direct memory access, low overhead).
    - **Weakness:** Requires **synchronization** (risk of race conditions).
2. **Message Passing**
    - **Strength:** Safer and simpler (**no shared variables**).
    - **Weakness:** Slower due to **kernel involvement and copying overhead**.

b) Discuss one drawback faced by multi-programmed OS architecture and

suggest possible way(s) to overcome it.

b) Drawback of multiprogrammed OS [2]

Drawback:

- **CPU scheduling complexity / possible starvation** (some processes may wait too long).

Solution:

- Use **better scheduling algorithms** (e.g., Round Robin, MLFQ, aging) to ensure fairness.

c) What is the purpose of system calls?

c) System calls

Purpose of system calls:

- Provide an **interface between user programs and OS kernel** to safely access system resources.

a) Explain data parallelism with an example.

Answer:
Data parallelism means dividing a large dataset into smaller parts and processing them **simultaneously on multiple processors**.

Example:
An array of 100 elements can be split into 10 parts, and each core computes the sum of its part in parallel.

b) You are developing a real-time embedded system for a safety-critical

application, such as an advanced driver assistance system (ADAS) for

autonomous vehicles. The system's primary goal is to process sensor

data, make critical decisions, and take actions in real-time to ensure the

safety of passengers and pedestrians. However, you may assume there's no limitation of computational resources on the project you are working on.

Based on the scenario, which multi-threading model would you recommend for implementation? Provide necessary justification.

Answer:
One-to-One threading model is recommended.

Justification:

- Each user thread maps to a **separate kernel thread**, allowing **true parallelism**.
- Suitable for **real-time, safety-critical systems** like ADAS where fast response is required.
- Blocking of one thread does not affect others, ensuring **reliability and responsiveness**.

c) A system has processes to execute of which 30% is serial. If the number of cores is decreased from 9 to 4, Explain the change in the performance.

Given 30% serial → 70% parallel

Speedup formula:

$$S = \frac{1}{(0.3 + \frac{0.7}{N})}$$

- For **9 cores:**

$$S_9 = \frac{1}{0.3 + 0.7/9} \approx \frac{1}{0.3778} \approx 2.65$$

- For **4 cores:**

$$S_4 = \frac{1}{0.3 + 0.7/4} = \frac{1}{0.475} \approx 2.10$$

Conclusion:
Performance **decreases** when cores are reduced (from ~2.65× to ~2.10×), due to the fixed serial portion limiting speedup.

a) How does the dual-mode operation contribute to the security and stability of an operating system? Provide an example of a privileged instructions and its significance.

Answer:
Dual-mode operation (user mode & kernel mode) improves **security and stability** by

restricting user programs from executing sensitive operations directly. Only the OS in kernel mode can perform critical tasks.

Example (privileged instruction):

- **Turn off interrupts**
  Significance: Prevents user programs from disrupting system control, ensuring safe and stable execution.

1) while running the application, she started

reading a book on OS and learned about a multithreading model which

allows only one thread to access the kernel at a time.Identify the multithreading model that Adhara learned of. Write down

one disadvantage of the model used and briefly explain the model that

overcomes that disadvantage.

Answer:

- **Model: Many-to-One model**

Disadvantage:

- If one thread makes a blocking system call, **all threads are blocked**.

Improved model:

- **One-to-One model**
  - Each user thread maps to a kernel thread, allowing **true parallelism** and preventing blocking of all threads.

A university library wants to ensure that only one student can use a

checkout machine at a time to prevent conflicts. Before checking out, a

student ensures if the “Checkout Key” is available. If yes, then the student

takes the key and inserts it into the designated slot on the checkout

machine, signaling that they are currently using it. This prevents others

from using the checkout machine. After completion, the student removes

the Checkout Key from the machine and returns it to its original location,

making it available for the next student to use.

a) Identify which synchronization method has been used here? Which synchronization problems can be solved by this method?

[3]

b) How do binary semaphores differ from counting semaphores? Provide an example of real-life implementation for each of the methods.

a) Synchronization method & problems [3]

Answer:

- The method used is a **Binary Semaphore (Mutex lock)**.

Explanation:

- The "Checkout Key" acts like a lock (value = 0 or 1).
- Only one student can hold the key at a time → ensures **mutual exclusion**.

Problems solved:

- **Critical Section Problem**
- **Race Condition prevention**
- Can be used in problems like **Producer–Consumer (with mutex)**

b) Binary vs Counting Semaphore [3]

Binary Semaphore:

- Value is **0 or 1**
- Used for **mutual exclusion (one resource)**
- **Example:** One printer → only one user can print at a time

Counting Semaphore:

- Value can be **0 to N**
- Used for **multiple identical resources**
- **Example:** Parking lot with 10 spaces → up to 10 cars can enter simultaneously

b) A program has multiple threads that need to perform a task which

involves similar computation on a large scale of data. Identify which kind of

parallelism can be applied in this scenario. Provide proper justification with

a real-life example of the scenario mentioned above.

Answer:
Data Parallelism

Justification:

- Same computation is applied to **large data sets simultaneously**.

Example:
Processing a large image where different threads apply filters to different parts of the image at the same time.

a) Explain how the two modes of the hardware enable the operating system to securely control user processes.

Answer:
Hardware supports **two modes: user mode and kernel mode**.

- User processes run in **user mode** with limited privileges.
- The OS runs in **kernel mode** with full access to hardware.
- Only kernel mode can execute **privileged instructions**.

Result:
This separation prevents user programs from harming the system, ensuring **security and stability**.

b) Explain the differences between Multiprogramming and Multiprocessing with examples.

Multiprogramming:

- Multiple programs in memory, **one CPU** switches between them.
- Goal: **maximize CPU utilization**.
- **Example:** Running browser + editor on a single-core system.

Multiprocessing:

- **Multiple CPUs/cores** execute processes simultaneously.
- Goal: **increase performance (parallelism)**.
- **Example:** Running tasks on a multi-core processor.

a) Describe what is the process control block, its contents, and how it is used. In particular, describe its role in context switching.

Answer:

PCB (Process Control Block):
A data structure used by the OS to **store information about a process./ track a process**

Contents:

- Process ID (PID)
- Process state (ready, running, waiting)
- Program counter
- CPU registers
- Memory management info
- I/O status and scheduling info

Role in context switching:

- When switching processes, the OS **saves the current process state in its PCB**.
- Then it **loads the next process state from its PCB**, allowing execution to resume correctly.

a) When is CPU scheduling required?

CPU scheduling is required when:

- **Multiple processes are in the ready state** (waiting for CPU).
- The OS must decide **which process gets the CPU next** to ensure **fairness, efficiency, and high CPU utilization**.
- Especially important in **multiprogramming and multitasking systems**.

Example:

- In a time-sharing system, many user processes compete for CPU, so scheduling decides the next process to run.

a) What is multiprogramming? What are the requirements for multiprogramming?

Definition:

- **Multiprogramming** is a technique where **multiple programs are loaded into memory** and executed **concurrently**, sharing the CPU to maximize utilization.

Requirements for Multiprogramming:

1. **Memory management** – enough memory to hold multiple programs.
2. **CPU scheduling** – decide which process runs next.
3. **I/O management** – handle multiple I/O requests efficiently.

4. **Process control** – track process states and execution.

c) Define system call? Explain how system call works using System Call Interface.

Definition:

- A **system call** is a **request made by a user program to the OS** to perform privileged operations on its behalf.

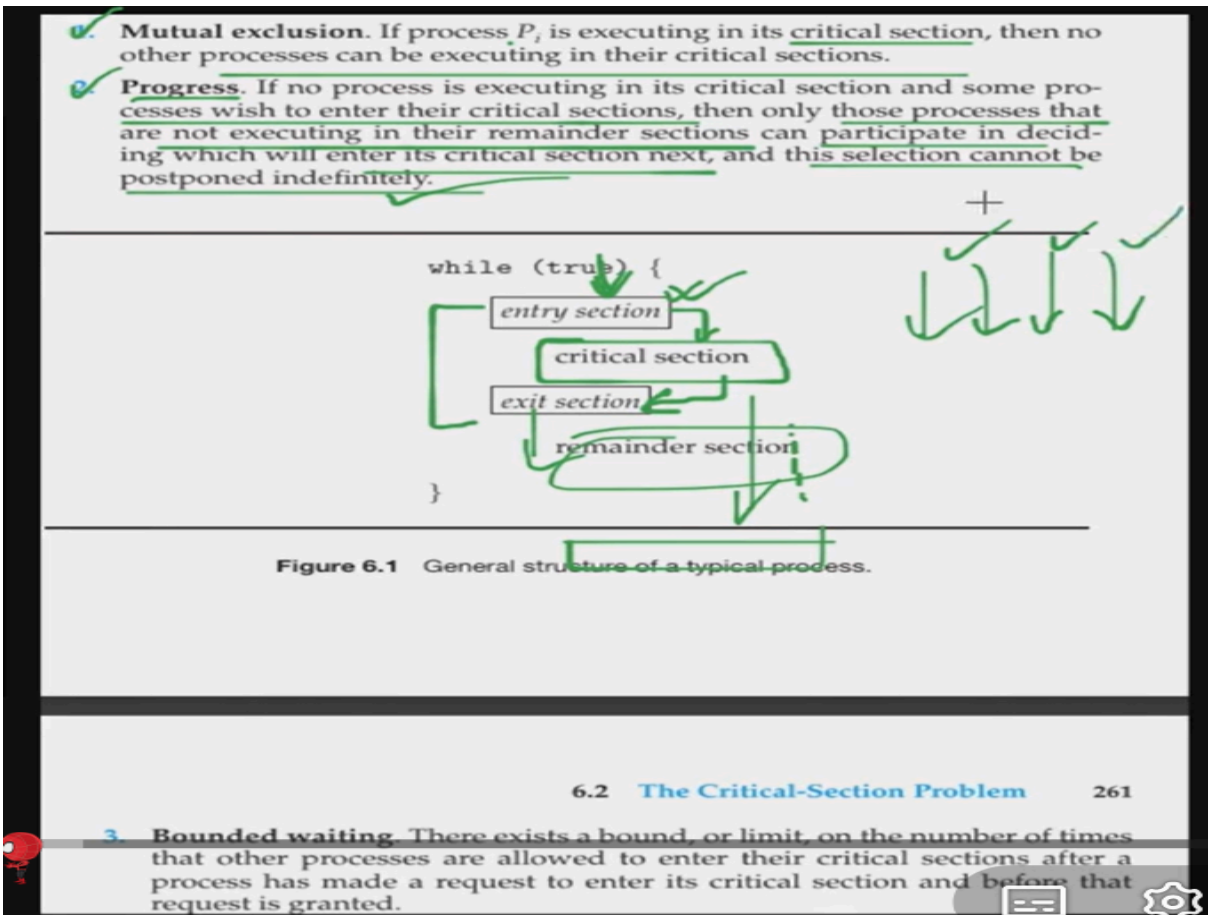
How it works (using System Call Interface):

1. User program executes a **library function** (e.g., read()).
2. The **System Call Interface (SCI)** transfers control to the **kernel**.
3. Kernel performs the requested operation (I/O, memory, process).
4. Result/status is returned to the **user program**.

a) Though we can increase the hardware resources as much as we want by money, still why do we implement the thread? Explain how the thread accelerates the computing power with an example.

Answer:

- Threads allow **parallel execution within a single process**, using **shared memory and resources**, which reduces **overhead** compared to creating multiple processes.
- **Acceleration example:**
  - A video encoding program divides the video into chunks.
  - Each thread processes a chunk **simultaneously on multiple cores**, making the program **run faster** without duplicating resources.



1. **Mutual exclusion**. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress**. If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

```
while (true) {
    entry section
        critical section
    exit section
        remainder section
}
```

Figure 6.1 General structure of a typical process.

3. **Bounded waiting**. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

How do binary semaphores differ from counting semaphores ? provide an example of real life implementation for each of the methods

=A binary semaphore can take only values 0 and 1 and is used for mutual exclusion, allowing only one process to access a critical section at a time. A counting semaphore can take values from 0 to n and is used to control access to multiple instances of a resource.

Example: A binary semaphore can be used as a lock for a single bathroom, while a counting semaphore can be used to manage a parking lot with multiple spaces.